SECURED SAFE VAULT API APPLICATION - REPORTS

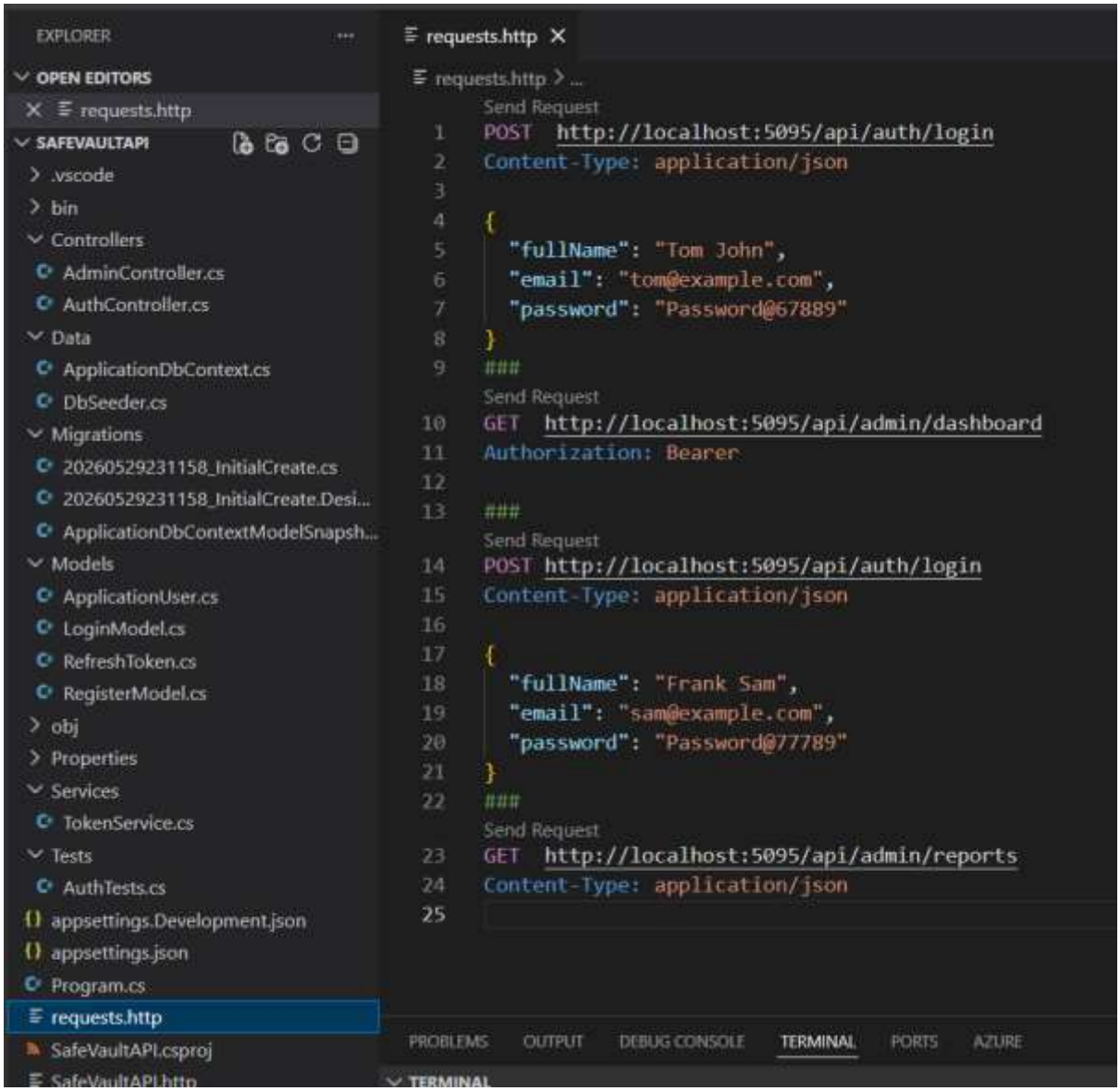
SafeVault is a secure web application designed to manage sensitive data, including user credentials and financial records.

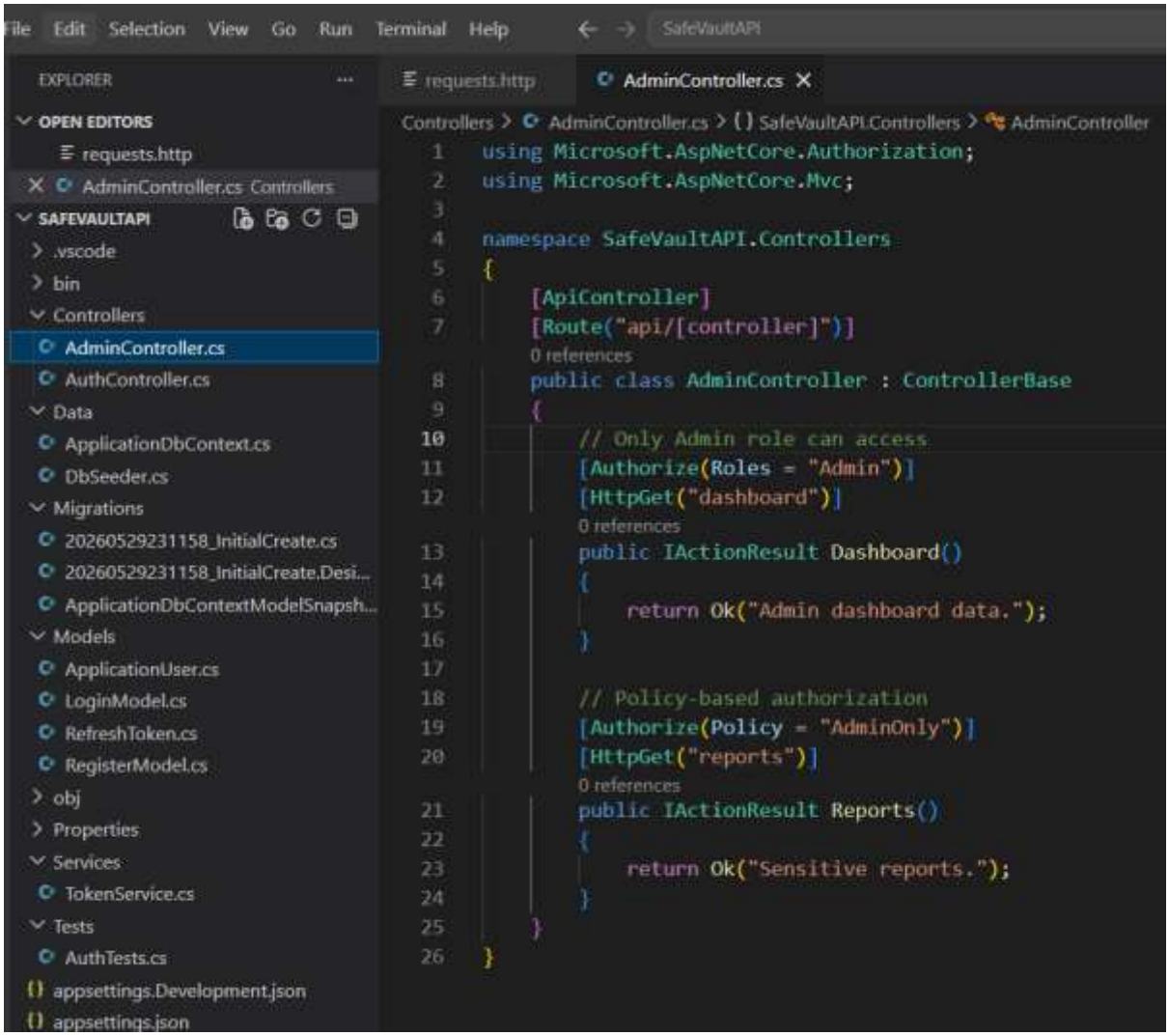
The AI was used to generate secure code for a web application, focusing on mitigating common vulnerabilities such as SQL injection and cross-site scripting (XSS).The codes were implemented and developed. Initially the application was not producing the expected outcomes so the AI was again used to debug and refine the codes. When the application worked fine, the desire to secure the application became indispensable to prevent malicious injections, eliminate SQL injection vulnerabilities and prevent XSS vulnerabilities through the injection of malicious scripts into user inputs. These requirements were achieved by the enforcing the use of parameterized queries to prevent SQL injection, the use of validation to validate user inputs to prevent malicious injections and securely handle user-provided data, such as login credentials or search inputs.

JWT validation is performed. JWT validation is like a checkpoint that confirms a token is trustworthy and hasn't been tampered with. Only tokens that pass this validation are allowed access. Another part of securing API endpoints with JWTs is using JWT middleware to automate token validation as a security gate for every request. Once set up, the middleware in ASP.NET Core checks each token's authenticity and expiration, blocking any requests with an invalid token.

The following shows the file structure of the SafeVaultAPI Application.

```
SAFEVAULTAPI/
│
├── .vscode/
│
├── bin/
│
├── obj/
│
├── Controllers/
│   ├── AdminController.cs
│   └── AuthController.cs
│
├── Data/
│   ├── ApplicationDbContext.cs
│   └── DbSeeder.cs
│
├── Migrations/
│   ├── 20260529231158_InitialCreate.cs
│   ├── 20260529231158_InitialCreate.Designer.cs
│   └── ApplicationDbContextModelSnapshot.cs
│
├── Models/
│   ├── ApplicationUser.cs
│   ├── LoginModel.cs
│   ├── RefreshToken.cs
│   └── RegisterModel.cs
│
├── Services/
│   └── TokenService.cs
│
├── Tests/
│   └── AuthTests.cs
│
├── Properties/
│
├── appsettings.json
├── appsettings.Development.json
│
├── Program.cs
├── requests.http
├── SafeVaultAPI.csproj
├── SafeVaultAPI.http
│
└── README / (not shown but usually here)
```


EXPLORER
OPEN EDITORS
requests.http
SAFEVAULTAPI
.vscode
bin
Controllers
AdminController.cs
AuthController.cs
Data
ApplicationDbContext.cs
DbSeeder.cs
Migrations
20260529231158_InitialCreate.cs
20260529231158_InitialCreate.Desi...
ApplicationDbContextModelSnapsh...
Models
ApplicationUser.cs
LoginModel.cs
RefreshToken.cs
RegisterModel.cs
obj
Properties
Services
TokenService.cs
Tests
AuthTests.cs
appsettings.Development.json
appsettings.json
Program.cs
requests.http
SafeVaultAPI.csproj
requests.http
Send Request
POST http://localhost:5095/api/auth/login
Content-Type: application/json
{
"fullName": "Tom John",
"email": "tom@example.com",
"password": "Password@67889"
}
###
Send Request
GET http://localhost:5095/api/admin/dashboard
Authorization: Bearer
###
Send Request
POST http://localhost:5095/api/auth/login
Content-Type: application/json
{
"fullName": "Frank Sam",
"email": "sam@example.com",
"password": "Password@77789"
}
###
Send Request
GET http://localhost:5095/api/admin/reports
Content-Type: application/json
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
AZURE
TERMINAL



The AuthController.cs (Security)

The program codes in the AuthController.cs file performs most of the security expected as shown below:

ASP.NET Identity Security

The biggest security feature comes from:

```
private readonly UserManager<ApplicationUser> _userManager;
```

ASP.NET Identity automatically provides several protections.

Password Hashing

During registration:

```
await _userManager.CreateAsync(user, model.Password);
```

The password is **not stored in plain text.**

ASP.NET Identity:

1. Generates a random salt.
2. Hashes the password.
3. Stores only the hash.

Example:

Instead of:

Password123!

Database stores something similar to:

```
AQAAAAEAACcQAAAAEM5...
```

Benefits:

- Database compromise does not reveal passwords directly.
- Rainbow table attacks are mitigated through salting.
- Modern hashing algorithms are used.

Password Verification

During login:

```
await _userManager.CheckPasswordAsync(user, model.Password);
```

The supplied password is hashed again and compared to the stored hash.

Benefits:

- Plain-text passwords never need to be retrieved.
- Secure comparison is performed by Identity.

---

2. Model Validation Protection

Both endpoints use:

```
if (!ModelState.IsValid)
    return BadRequest(ModelState);
```

Assuming the models contain validation attributes:

```
[Required]
[EmailAddress]
```

or

```
[StringLength]
```

This protects against:

- ❖ Missing required fields (this is presence check)
- ❖ Invalid email formats
- ❖ Invalid data types
- ❖ Basic malformed requests

Duplicate Account Protection

Before registration:

```
var existingUser =
    await _userManager.FindByEmailAsync(model.Email);

if (existingUser != null)
    return BadRequest("Email already exists.");
```

The block of codes above Prevents:

- Multiple accounts with same email
- Identity confusion
- Account duplication

---

4. JWT Security Features

The JWT method contains multiple security mechanisms.

Digital Signature

```
var creds = new SigningCredentials(
    key,SecurityAlgorithms.HmacSha256);
```

Uses:

HMAC-SHA256to sign the token. This prevents attackers from changing claims.

```
var key = new SymmetricSecurityKey(
    Encoding.UTF8.GetBytes(_configuration["Jwt:Key"]!));

var creds = new SigningCredentials(
    key,
    SecurityAlgorithms.HmacSha256);

var expires = DateTime.UtcNow.AddMinutes(
    Convert.ToDouble(_configuration["Jwt:DurationInMinutes"]));
```

Without signing:

```
{
   "role":"User"
}
```

could be changed to:

```
{
   "role":"Admin"
}
```

With signing:

- Signature becomes invalid.
- Token is rejected.

Secret Key Protection

```
new SymmetricSecurityKey(Encoding.UTF8.GetBytes(
        _configuration["Jwt:Key"]!));
```

The secret key is externalized.

Benefits:

- Key not hardcoded.
- Easier key rotation.
- Different environments can use different keys.

```
"Jwt": {
    "Key": "MyVeryStrongSecretKeyForJwtAuthentication2026!",
    "Issuer": "SafeVaultAPI",
    "Audience": "SafeVaultAPIUsers",
    "DurationInMinutes": 15
},
```

Expiration Protection

```
var expires = DateTime.UtcNow.AddMinutes(
    Convert.ToDouble(_configuration["Jwt:DurationInMinutes"]));

var token = new JwtSecurityToken(
    issuer: _configuration["Jwt:Issuer"],
    audience: _configuration["Jwt:Audience"],
    claims: claims,
    expires: expires,
    signingCredentials: creds);
```

Benefits:

- Limits token lifetime.
- Reduces risk from stolen tokens.

Example:

Token valid:
12:00 -> 13:00

After expiration:

401 Unauthorized will be displayed. This is a protection mechanism.

Issuer Validation

issuer:
_configuration["Jwt:Issuer"]

Used to validate:

Who issued the token and hence prevents accepting tokens from other issuers.

```
var token = new JwtSecurityToken(
    issuer: _configuration["Jwt:Issuer"],
    audience: _configuration["Jwt:Audience"],
    claims: claims,
    expires: expires,
    signingCredentials: creds);
```

Audience Validation

audience:
_configuration["Jwt:Audience"]

Defines:

Who is allowed to use the token? This prevents token reuse across applications.

Example:

Token for:

SafeVaultAPI should not work in: PayrollAPI

---

5. Role-Based Authorization Support

Roles are retrieved:

```
var roles =  await _userManager.GetRolesAsync(user);
```

and added as claims:

```
claims.Add(new Claim(ClaimTypes.Role, role));
```

This enables:

```
[Authorize(Roles="Admin")]
```

Example:

```
{
  "role": "Admin"
}
```

can later be used for access control.

---

6. Logging and Auditing

Successful registration:

```
_logger.LogInformation(...)
```

Failed login:

```
_logger.LogWarning(...)
```

Provides:

- Audit trail
- Security monitoring
- Attack investigation

Example log:

Login failed. Invalid password:
john@email.com

Useful for:

- Brute force detection
- Account abuse detection
- Incident response

```
    // ==========================
    // Login User
    // ==========================
    [HttpPost("login")]
    0 references
    public async Task<IActionResult> Login(LoginModel model)
    {
        if (!ModelState.IsValid)
            return BadRequest(ModelState);

        var user =
            await _userManager.FindByEmailAsync(model.Email);

        if (user == null)
        {
            _logger.LogWarning(
                $"Login failed. User not found: {model.Email}");

            return Unauthorized("Invalid credentials.");
        }

        var passwordValid =
            await _userManager.CheckPasswordAsync(
                user,
                model.Password);

        if (!passwordValid)
        {
            _logger.LogWarning(
                $"Login failed. Invalid password: {model.Email}");

            return Unauthorized("Invalid credentials.");
        }
```

4. Login Enumeration Through Logs

Logs reveal:

User not found

vs

Invalid password

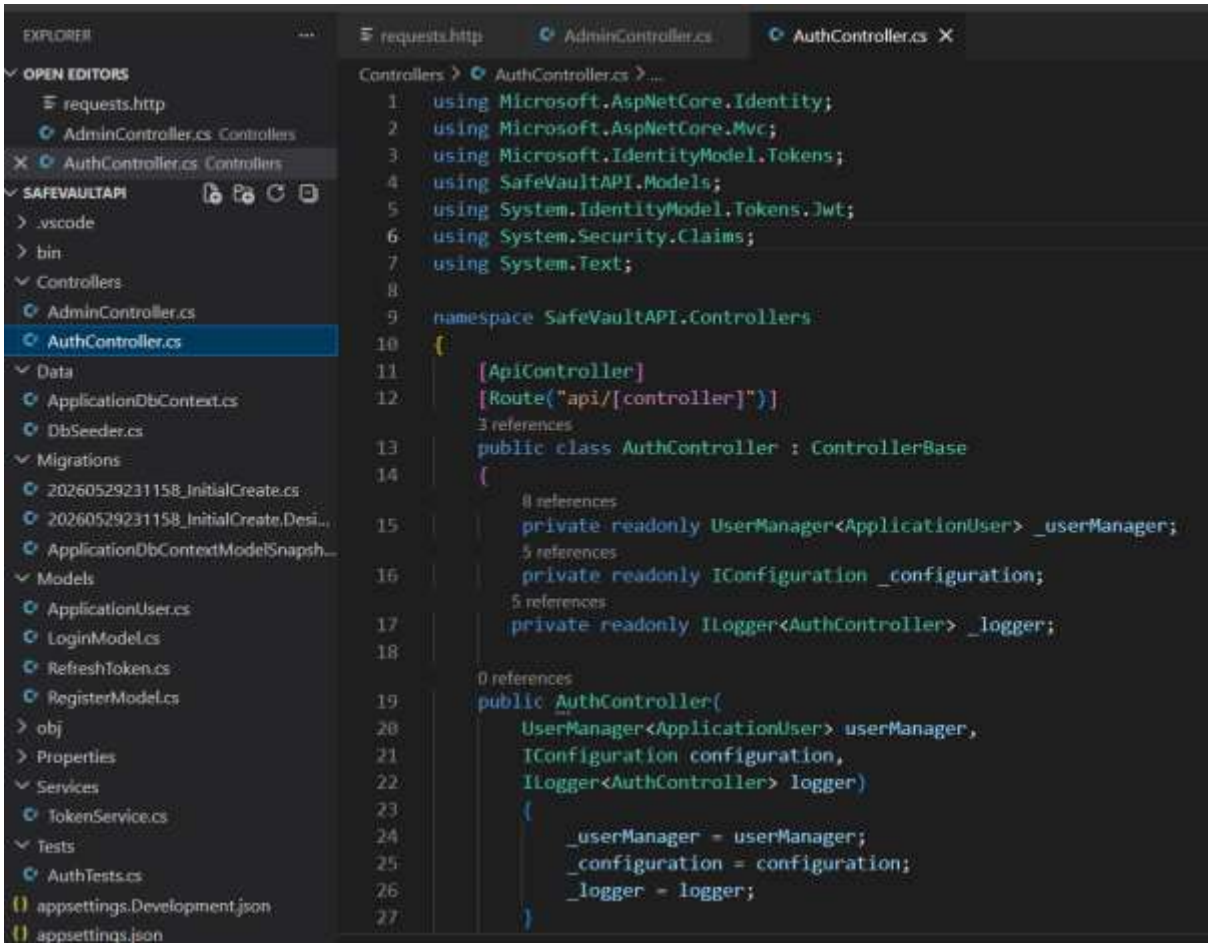
Although the API response is identical:

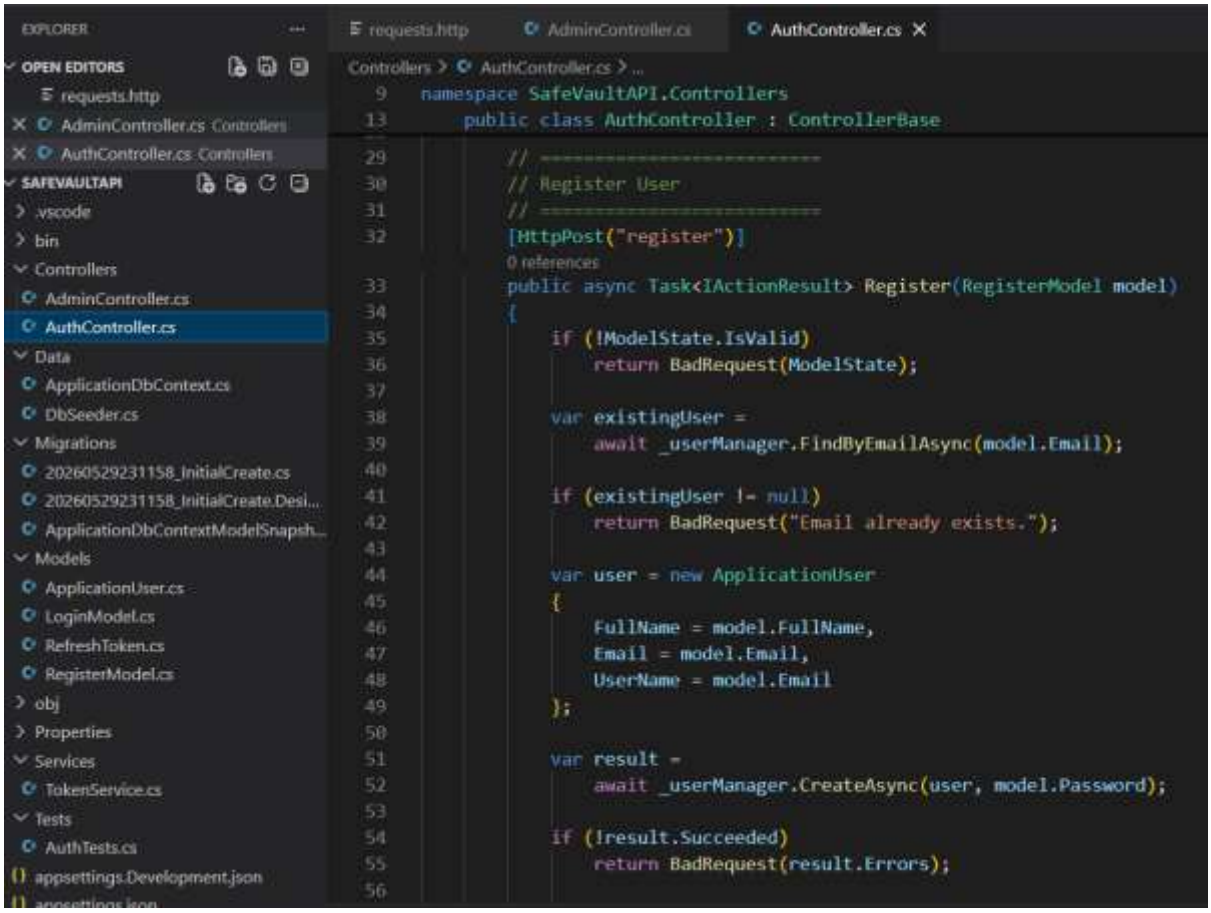
Unauthorized("Invalid credentials.")

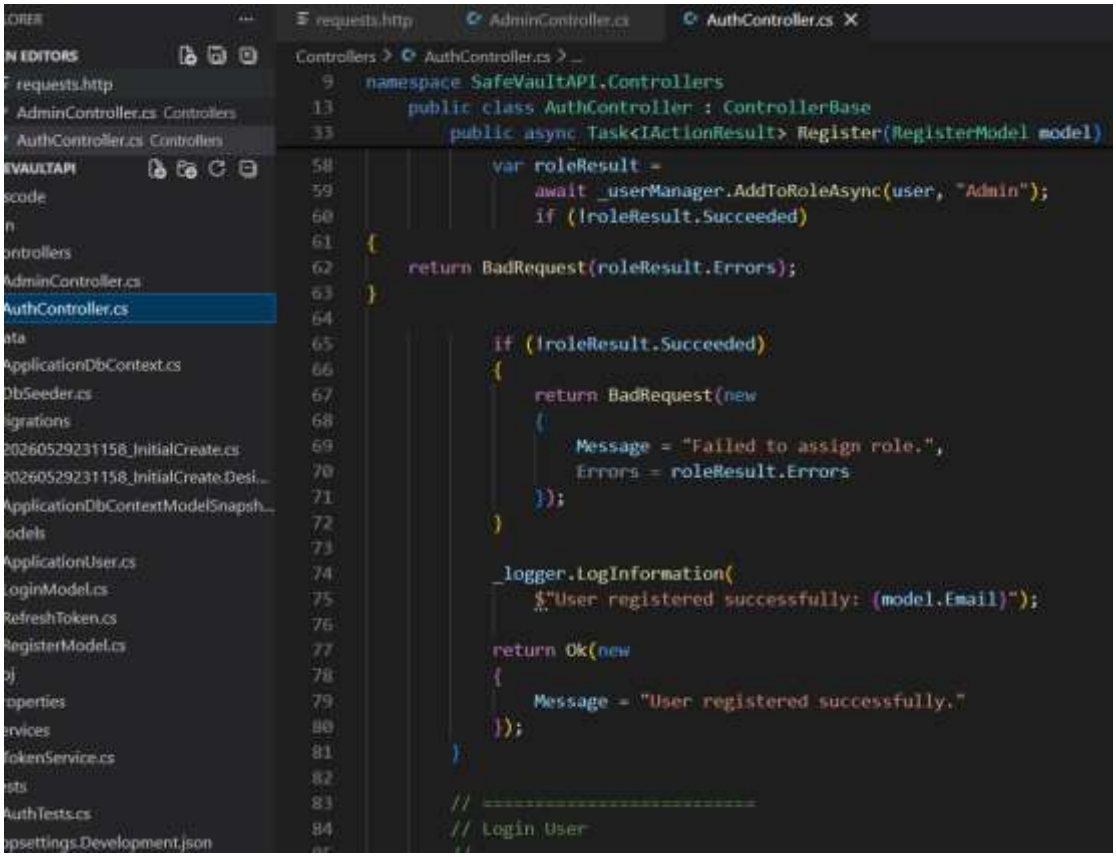
the logs reveal account existence.

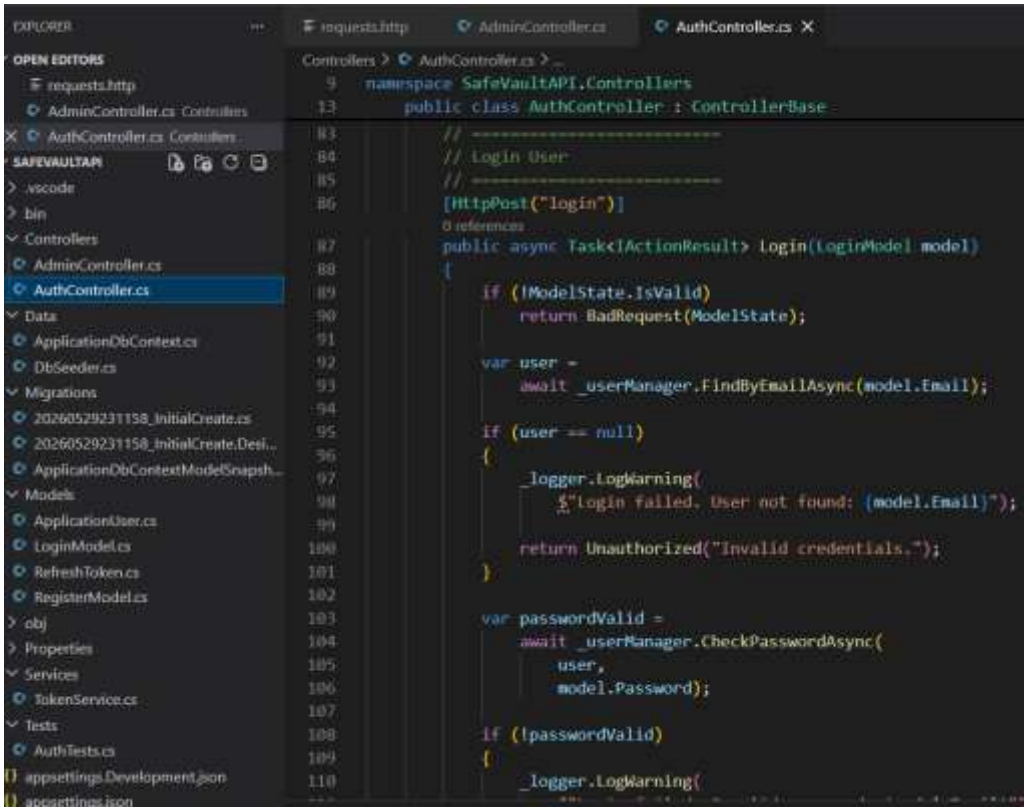
This may be acceptable for internal logs but should be protected carefully.

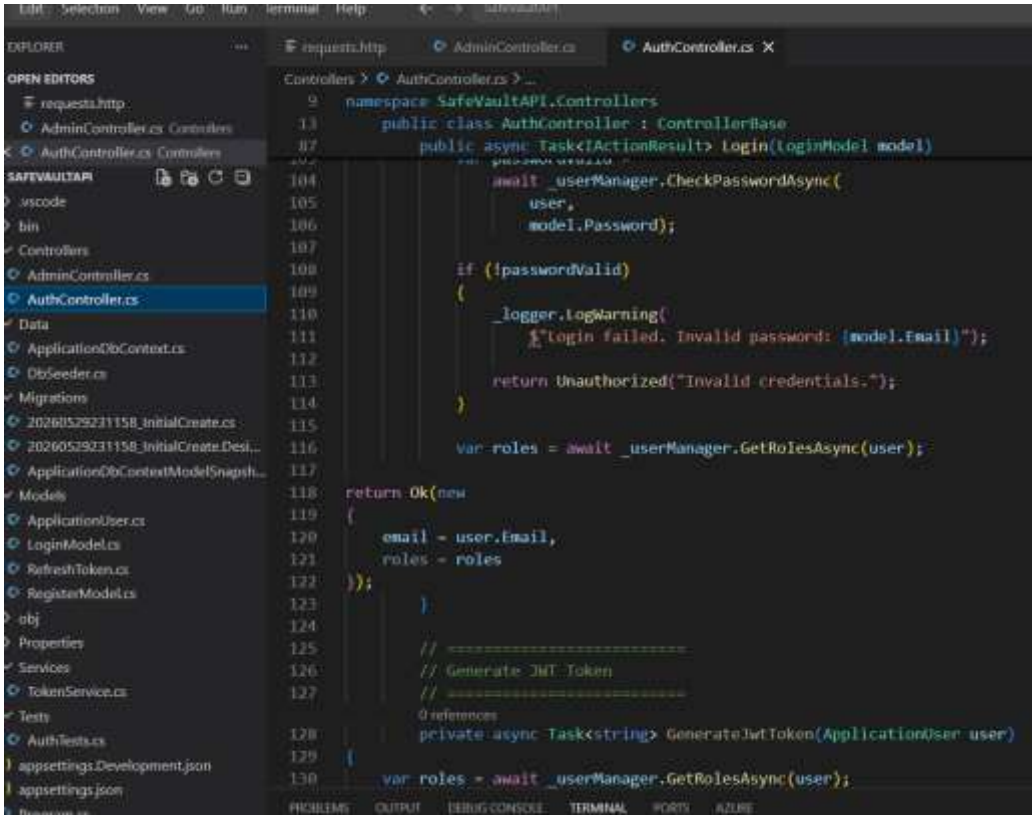
FULL IMPLEMENTATION OF THE SECURITY WITHIN AuthController.cs

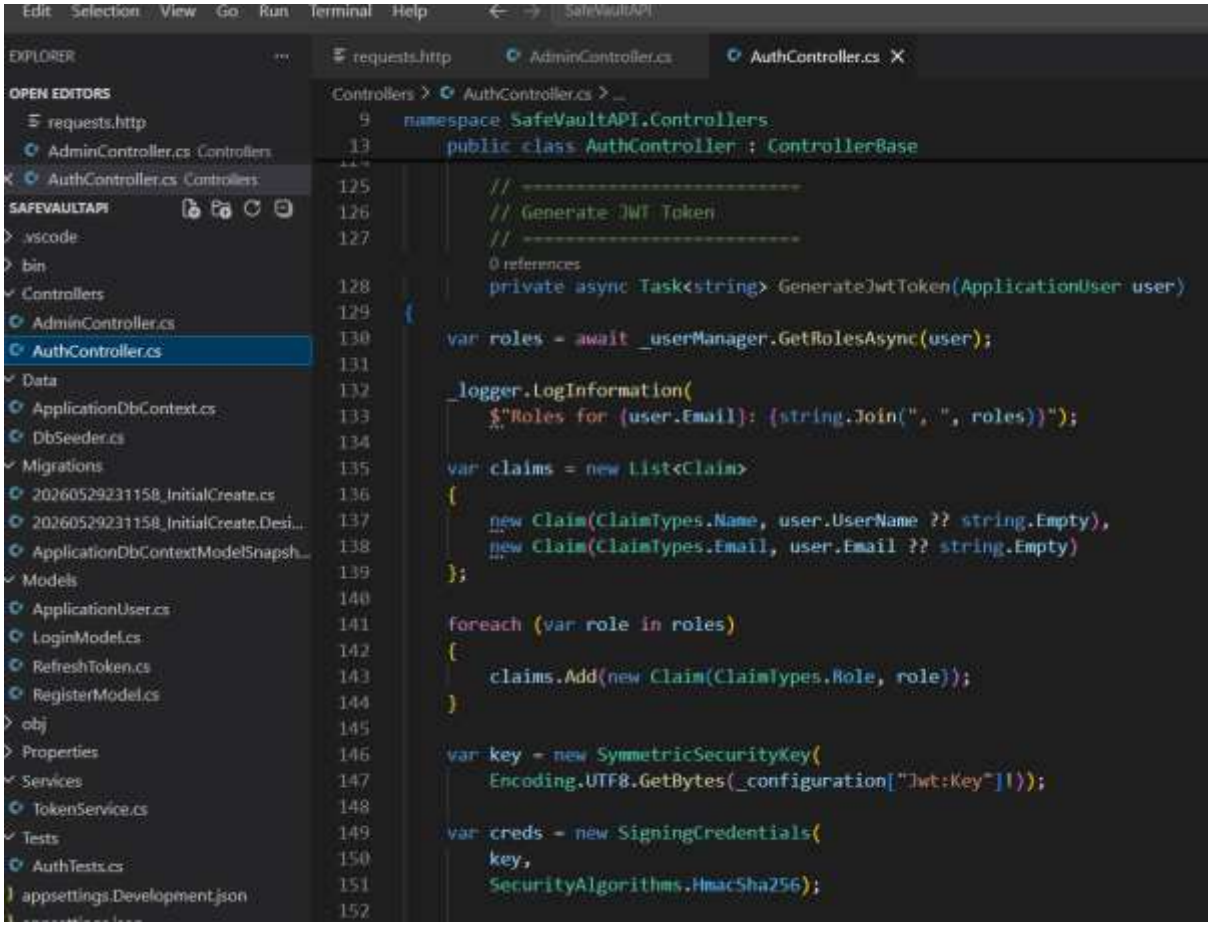


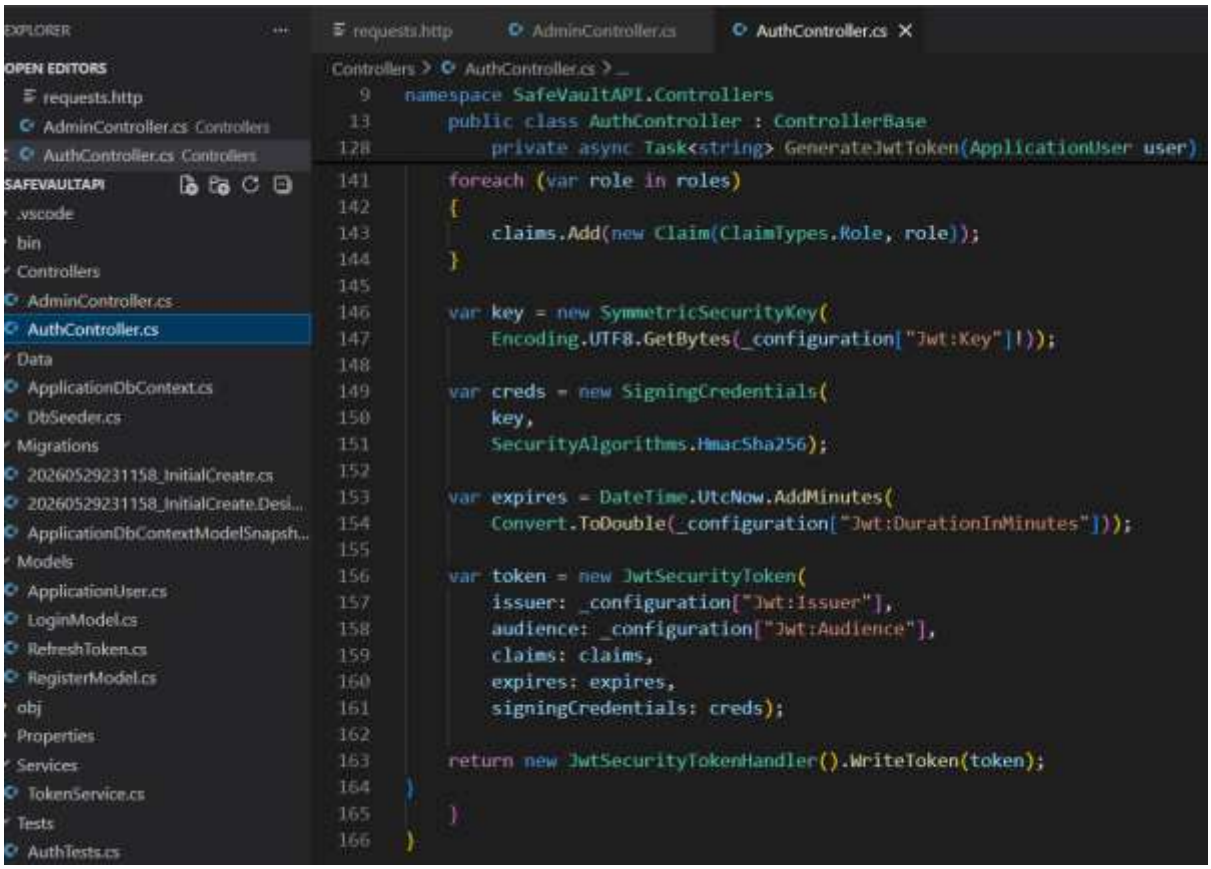

EXPLORER
OPEN EDITORS
requests.http
AdminController.cs Controllers
AuthController.cs Controllers
SAFEVAULTAPI
.vscode
bin
Controllers
AdminController.cs
AuthController.cs
Data
ApplicationDbContext.cs
DbSeeder.cs
Migrations
20260529231158_InitialCreate.cs
20260529231158_InitialCreate.Desi...
ApplicationDbContextModelSnapsh...
Models
ApplicationUser.cs
LoginModel.cs
RefreshToken.cs
RegisterModel.cs
obj
Properties
Services
TokenService.cs
Tests
AuthTests.cs
appsettings.Development.json
requests.http
AdminController.cs
AuthController.cs
Controllers > AuthController.cs > ...
namespace SafeVaultAPI.Controllers
public class AuthController : ControllerBase
// ==========
// Register User
// ==========
[HttpPost("register")]
0 references
public async Task<IActionResult> Register(RegisterModel model)
{
if (!ModelState.IsValid)
return BadRequest(ModelState);
var existingUser =
await _userManager.FindByEmailAsync(model.Email);
if (existingUser != null)
return BadRequest("Email already exists.");
var user = new ApplicationUser
{
FullName = model.FullName,
Email = model.Email,
UserName = model.Email
};
var result =
await _userManager.CreateAsync(user, model.Password);
if (!result.Succeeded)
return BadRequest(result.Errors);


requests.http
AdminController.cs
AuthController.cs
Controllers > AuthController.cs > ...
namespace SafeVaultAPI.Controllers
public class AuthController : ControllerBase
public async Task<IActionResult> Register(RegisterModel model)
var roleResult =
await _userManager.AddToRoleAsync(user, "Admin");
if (!roleResult.Succeeded)
{
return BadRequest(roleResult.Errors);
}
if (!roleResult.Succeeded)
{
return BadRequest(new
{
Message = "Failed to assign role.",
Errors = roleResult.Errors
});
}
_logger.LogInformation(
$"User registered successfully: {model.Email}");
return Ok(new
{
Message = "User registered successfully."
});
}
// ==========
// Login User


requests.http
AdminController.cs
AuthController.cs
Controllers > AuthController.cs > ...
namespace SafeVaultAPI.Controllers
public class AuthController : ControllerBase
// Login User
[HttpPost("login")]
public async Task<IActionResult> Login(LoginModel model)
if (!ModelState.IsValid)
return BadRequest(ModelState);
var user =
await _userManager.FindByEmailAsync(model.Email);
if (user == null)
_logger.LogWarning(
$"Login failed. User not found: {model.Email}");
return Unauthorized("Invalid credentials.");
var passwordValid =
await _userManager.CheckPasswordAsync(
user,
model.Password);
if (!passwordValid)
_logger.LogWarning(

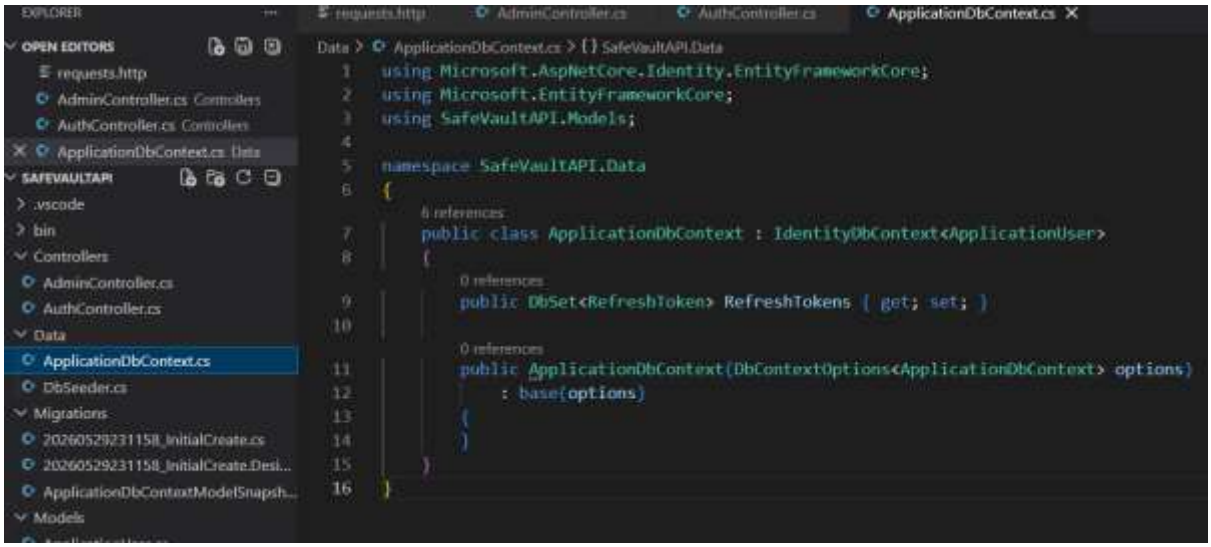

requests.http
AdminController.cs
AuthController.cs
Controllers > AuthController.cs > ...
namespace SafeVaultAPI.Controllers
public class AuthController : ControllerBase
public async Task<IActionResult> Login(LoginModel model)
await _userManager.CheckPasswordAsync(
user,
model.Password);
if (!passwordValid)
_logger.LogWarning(
$"Login failed. Invalid password: {model.Email}");
return Unauthorized("Invalid credentials.");
var roles = await _userManager.GetRolesAsync(user);
return Ok(new
{
email = user.Email,
roles = roles
});
// Generate JWT Token
private async Task<string> GenerateJwtToken(ApplicationUser user)
var roles = await _userManager.GetRolesAsync(user);

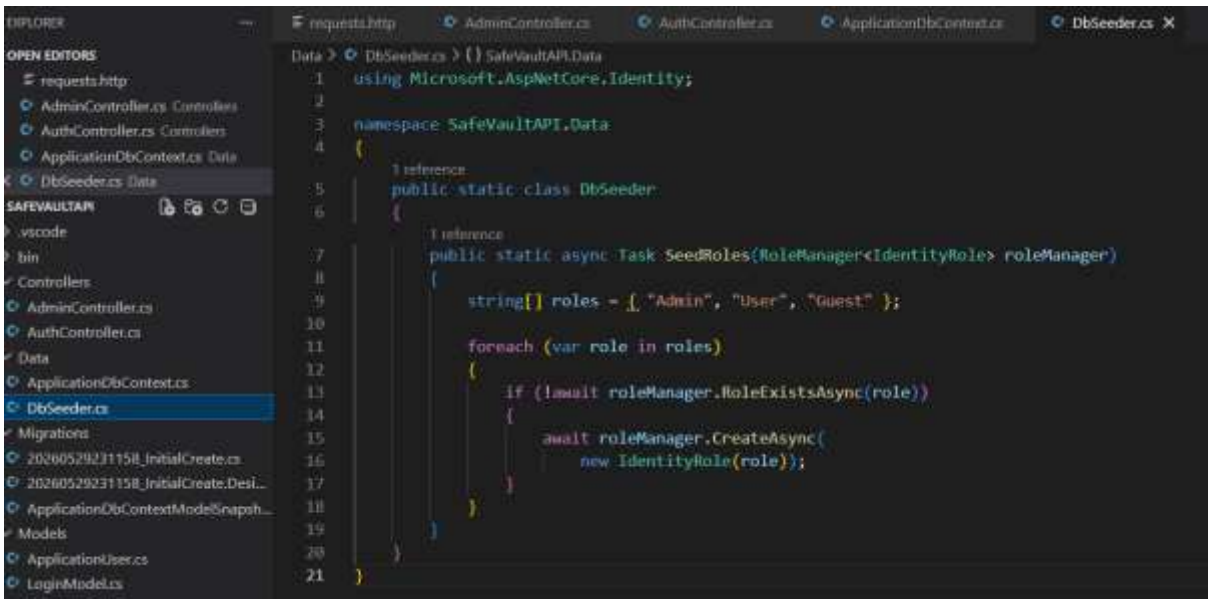

Edit Selection View Go Run Terminal Help
EXPLORER
OPEN EDITORS
requests.http
AdminController.cs Controllers
AuthController.cs Controllers
SAFEVAULTAPI
.vscode
bin
Controllers
AdminController.cs
AuthController.cs
Data
ApplicationDbContext.cs
DbSeeder.cs
Migrations
20260529231158_InitialCreate.cs
20260529231158_InitialCreate.Desi...
ApplicationDbContextModelSnapsh...
Models
ApplicationUser.cs
LoginModel.cs
RefreshToken.cs
RegisterModel.cs
obj
Properties
Services
TokenService.cs
Tests
AuthTests.cs
appsettings.Development.json
requests.http
AdminController.cs
AuthController.cs
Controllers > AuthController.cs > ...
9 namespace SafeVaultAPI.Controllers
13 public class AuthController : ControllerBase
125 // ------------------------
126 // Generate JWT Token
127 // ------------------------
0 references
128 private async Task<string> GenerateJwtToken(ApplicationUser user)
129 {
130 var roles = await _userManager.GetRolesAsync(user);
131
132 _logger.LogInformation(
133 $"Roles for {user.Email}: {string.Join(", ", roles)}");
134
135 var claims = new List<Claim>
136 {
137 new Claim(ClaimTypes.Name, user.UserName ?? string.Empty),
138 new Claim(ClaimTypes.Email, user.Email ?? string.Empty)
139 };
140
141 foreach (var role in roles)
142 {
143 claims.Add(new Claim(ClaimTypes.Role, role));
144 }
145
146 var key = new SymmetricSecurityKey(
147 Encoding.UTF8.GetBytes(_configuration["Jwt:Key"]!));
148
149 var creds = new SigningCredentials(
150 key,
151 SecurityAlgorithms.HmacSha256);
152


EXPLORER
OPEN EDITORS
requests.http
AdminController.cs Controllers
AuthController.cs Controllers
SAFEVAULTAPI
.vscode
bin
Controllers
AdminController.cs
AuthController.cs
Data
ApplicationDbContext.cs
DbSeeder.cs
Migrations
20260529231158_InitialCreate.cs
20260529231158_InitialCreate.Desi...
ApplicationDbContextModelSnapsh...
Models
ApplicationUser.cs
LoginModel.cs
RefreshToken.cs
RegisterModel.cs
obj
Properties
Services
TokenService.cs
Tests
AuthTests.cs
requests.http
AdminController.cs
AuthController.cs
Controllers > AuthController.cs > ...
9 namespace SafeVaultAPI.Controllers
13 public class AuthController : ControllerBase
128 private async Task<string> GenerateJwtToken(ApplicationUser user)
141 foreach (var role in roles)
142 {
143 claims.Add(new Claim(ClaimTypes.Role, role));
144 }
145
146 var key = new SymmetricSecurityKey(
147 Encoding.UTF8.GetBytes(_configuration["Jwt:Key"]!));
148
149 var creds = new SigningCredentials(
150 key,
151 SecurityAlgorithms.HmacSha256);
152
153 var expires = DateTime.UtcNow.AddMinutes(
154 Convert.ToDouble(_configuration["Jwt:DurationInMinutes"]));
155
156 var token = new JwtSecurityToken(
157 issuer: _configuration["Jwt:Issuer"],
158 audience: _configuration["Jwt:Audience"],
159 claims: claims,
160 expires: expires,
161 signingCredentials: creds);
162
163 return new JwtSecurityTokenHandler().WriteToken(token);
164 }
165 }
166 }

APPLICATION DB CONTEXT

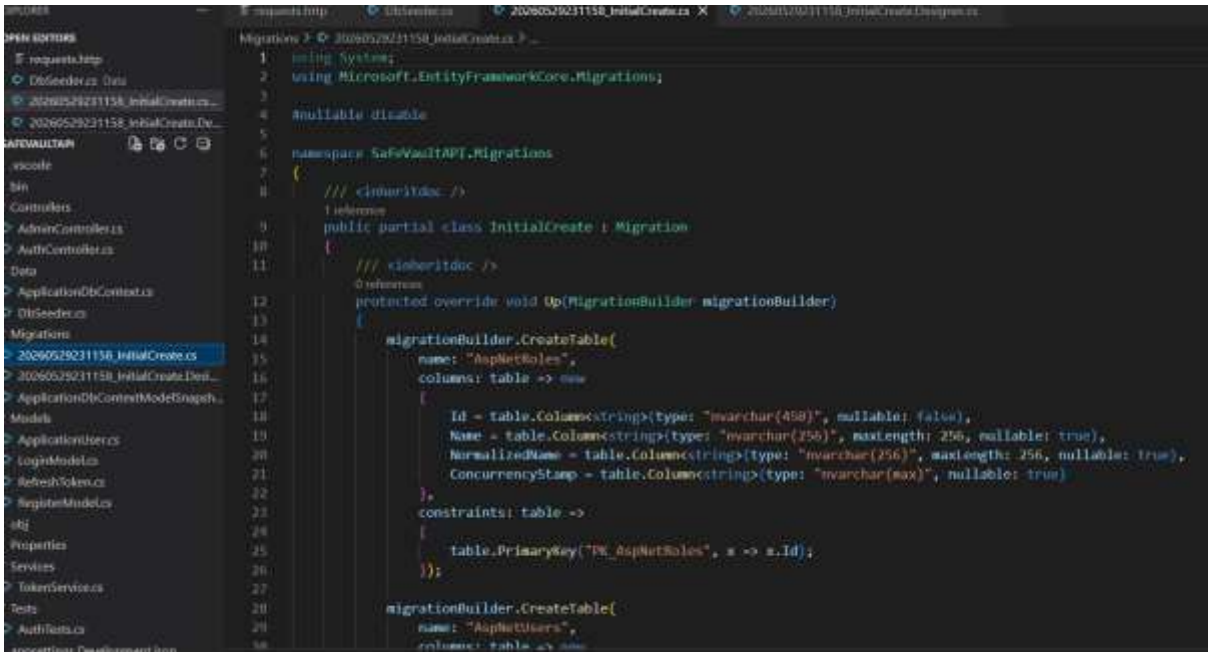
ApplicationDbContext : is the class that acts as the application's connection to the database.





MIGRATIONS

A migration **in Entity Framework Core is a mechanism for** keeping the database schema synchronized with your C# model classes**.**



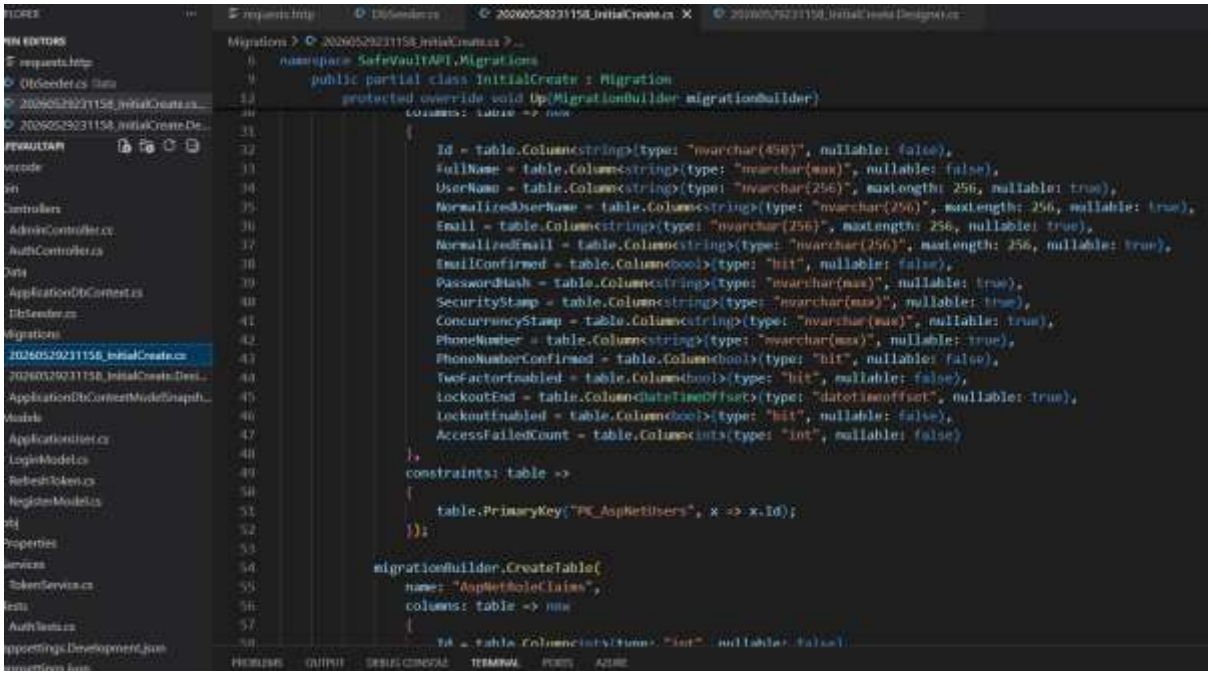
```
using System;
using Microsoft.EntityFrameworkCore.Migrations;

#nullable disable

namespace SafeVaultAPI.Migrations
{
    /// <inheritdoc />
    public partial class InitialCreate : Migration
    {
        /// <inheritdoc />
        protected override void Up(MigrationBuilder migrationBuilder)
        {
            migrationBuilder.CreateTable(
                name: "AspNetRoles",
                columns: table => new
                {
                    Id = table.Column<string>(type: "nvarchar(450)", nullable: false),
                    Name = table.Column<string>(type: "nvarchar(256)", maxLength: 256, nullable: true),
                    NormalizedName = table.Column<string>(type: "nvarchar(256)", maxLength: 256, nullable: true),
                    ConcurrencyStamp = table.Column<string>(type: "nvarchar(max)", nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_AspNetRoles", x => x.Id);
                });

            migrationBuilder.CreateTable(
                name: "AspNetUsers",
```

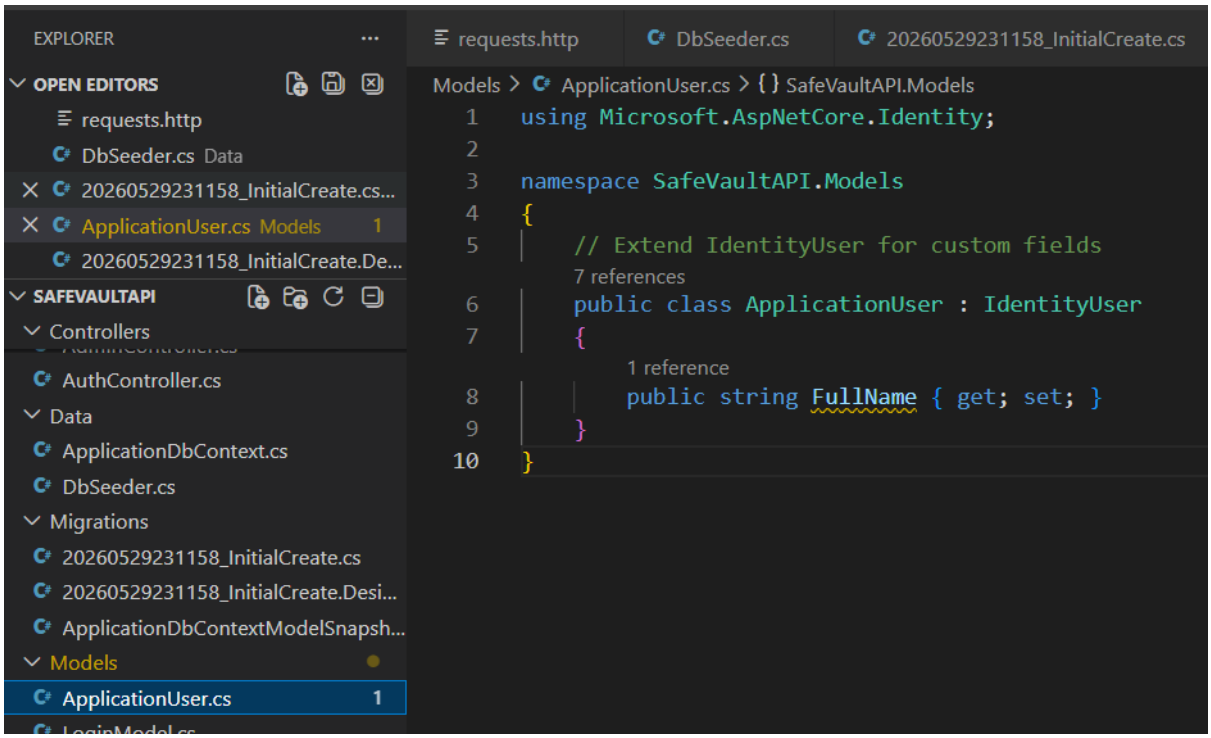
Creating the database entities and attributes based on the C# OOP models created.



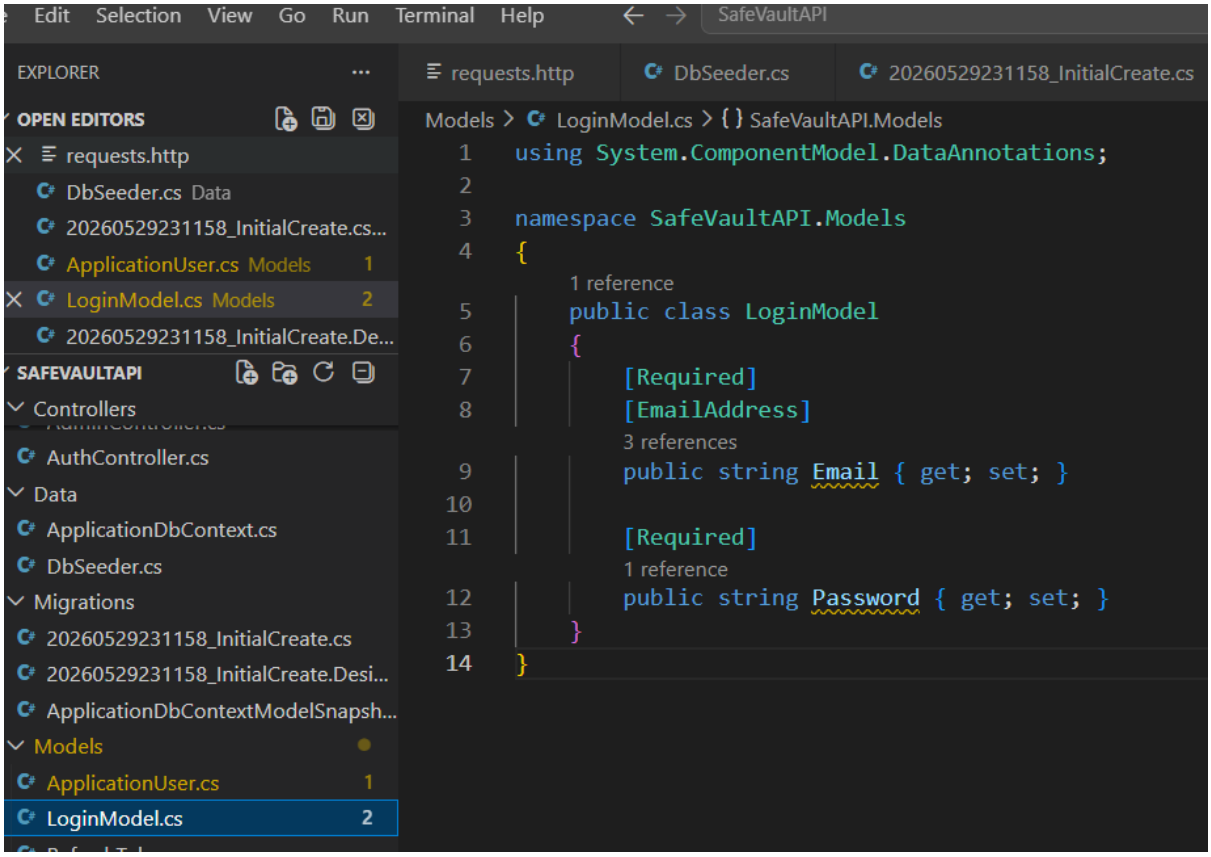
```
namespace SafeVaultAPI.Migrations
    public partial class InitialCreate : Migration
        protected override void Up(MigrationBuilder migrationBuilder)
            {
                Id = table.Column<string>(type: "nvarchar(450)", nullable: false),
                FullName = table.Column<string>(type: "nvarchar(max)", nullable: false),
                UserName = table.Column<string>(type: "nvarchar(256)", maxLength: 256, nullable: true),
                NormalizedUserName = table.Column<string>(type: "nvarchar(256)", maxLength: 256, nullable: true),
                Email = table.Column<string>(type: "nvarchar(256)", maxLength: 256, nullable: true),
                NormalizedEmail = table.Column<string>(type: "nvarchar(256)", maxLength: 256, nullable: true),
                EmailConfirmed = table.Column<bool>(type: "bit", nullable: false),
                PasswordHash = table.Column<string>(type: "nvarchar(max)", nullable: true),
                SecurityStamp = table.Column<string>(type: "nvarchar(max)", nullable: true),
                ConcurrencyStamp = table.Column<string>(type: "nvarchar(max)", nullable: true),
                PhoneNumber = table.Column<string>(type: "nvarchar(max)", nullable: true),
                PhoneNumberConfirmed = table.Column<bool>(type: "bit", nullable: false),
                TwoFactorEnabled = table.Column<bool>(type: "bit", nullable: false),
                LockoutEnd = table.Column<DateTimeOffset>(type: "datetimeoffset", nullable: true),
                LockoutEnabled = table.Column<bool>(type: "bit", nullable: false),
                AccessFailedCount = table.Column<int>(type: "int", nullable: false)
            },
            constraints: table =>
            {
                table.PrimaryKey("PK_AspNetUsers", x => x.Id);
            });

        migrationBuilder.CreateTable(
            name: "AspNetRoleClaims",
            columns: table => new
```

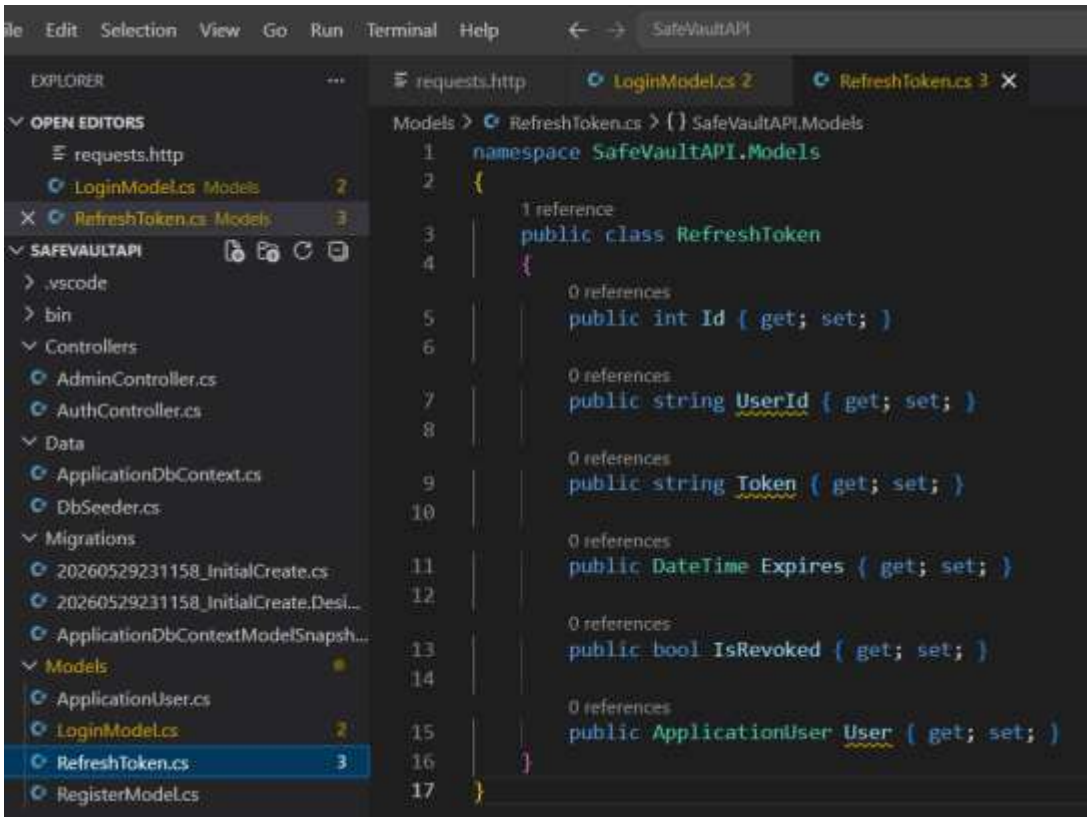
Here the IdentityUser class inherits from the ApplicationUser.



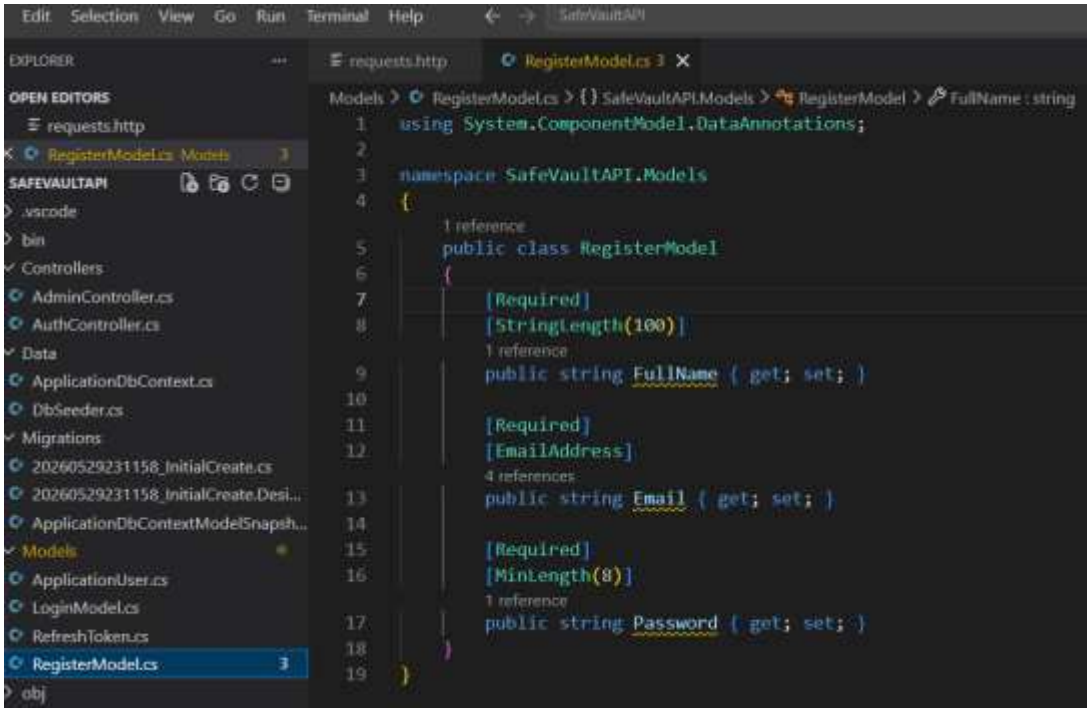
Here is the Login Model used to generate the database components for emails and the password.



This is the model class that creates the token

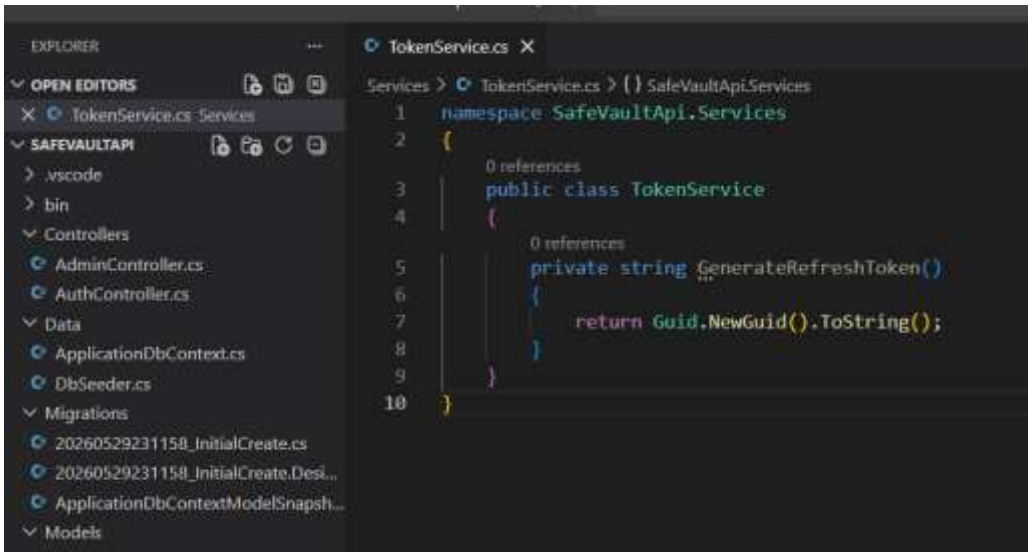


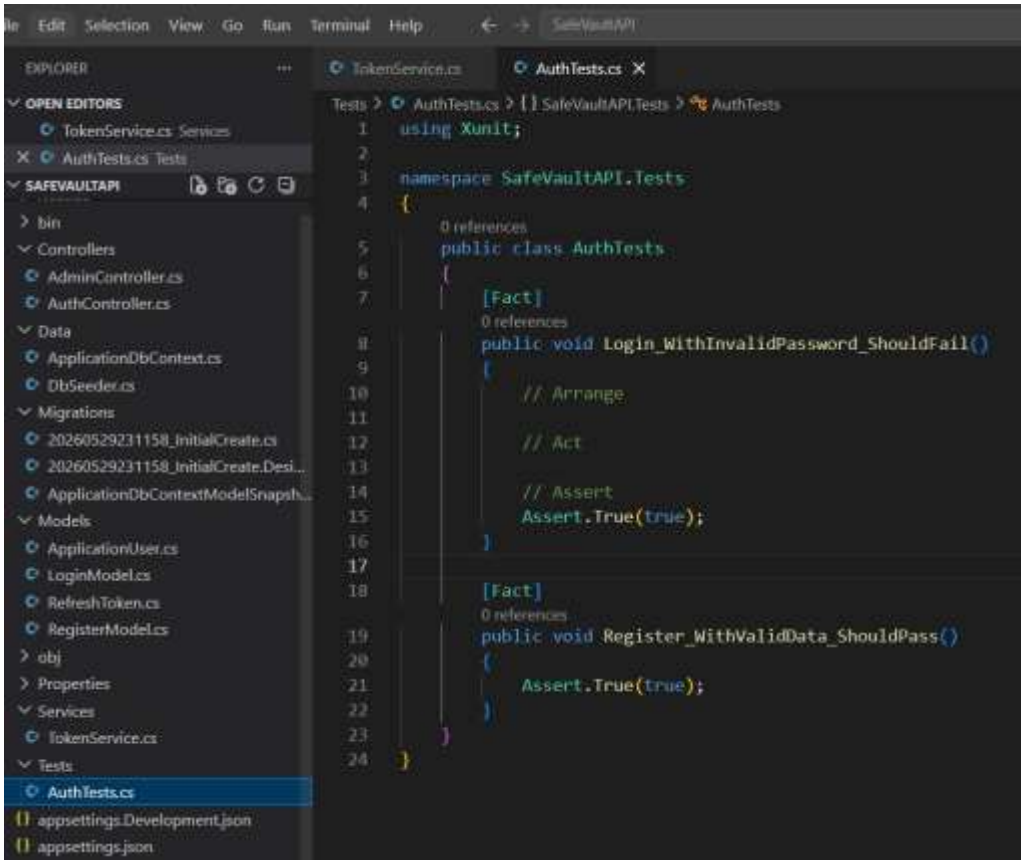
Here is the new user registration model; each attribute is validated for security reasons. This model is also used by migration to create the database objects.



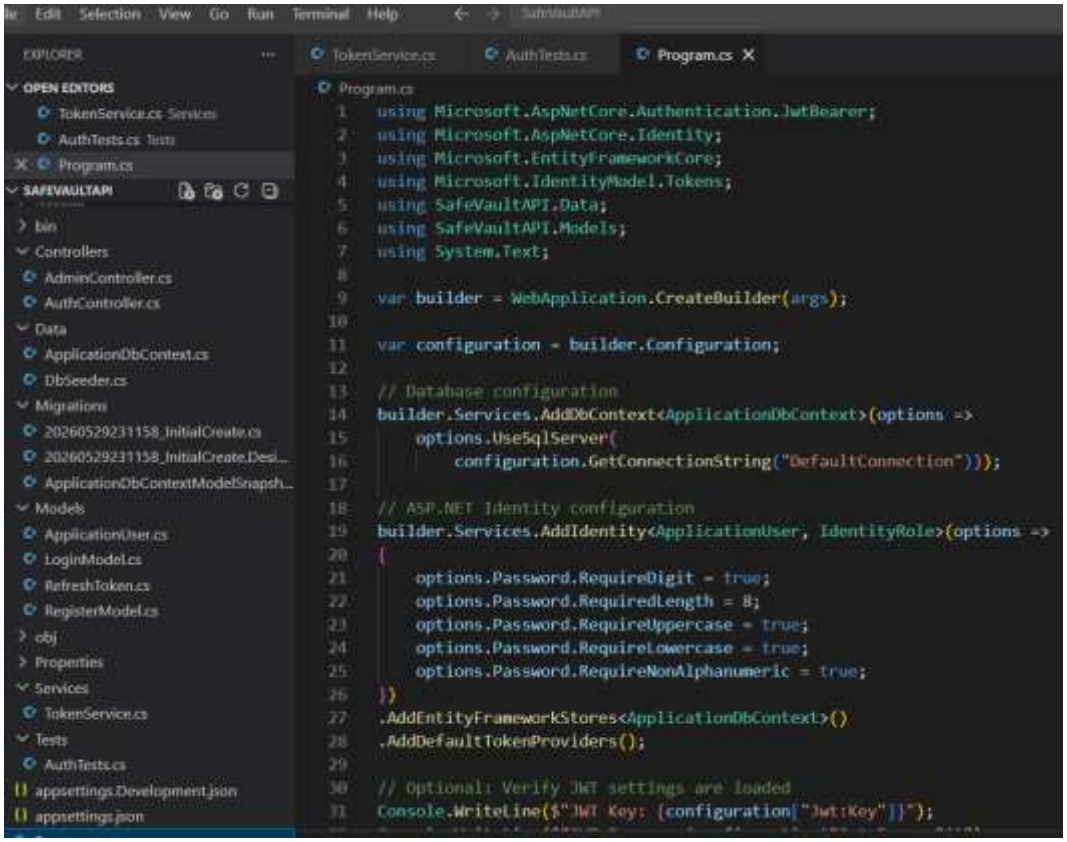
This program defines a very simple TokenService class with one private method:

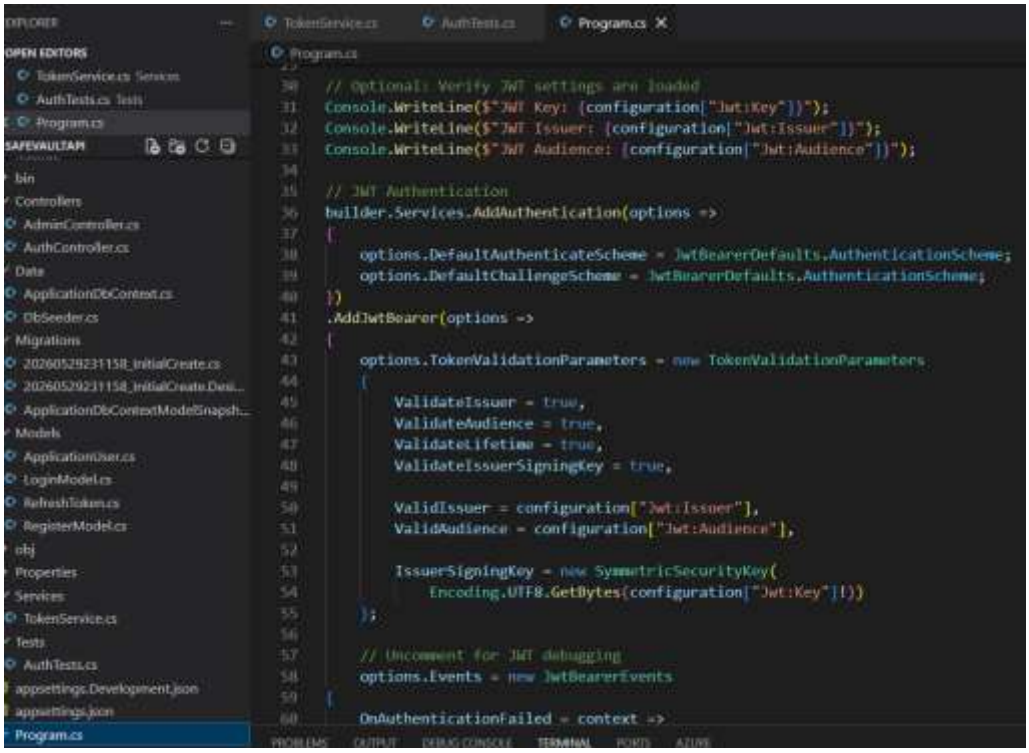
The function "GenerateRefreshToken()" creates and returns a new GUID (Globally Unique Identifier) as a string. An example is 'f47ac10b-58cc-4372-a567-0e02b2c3d479'.



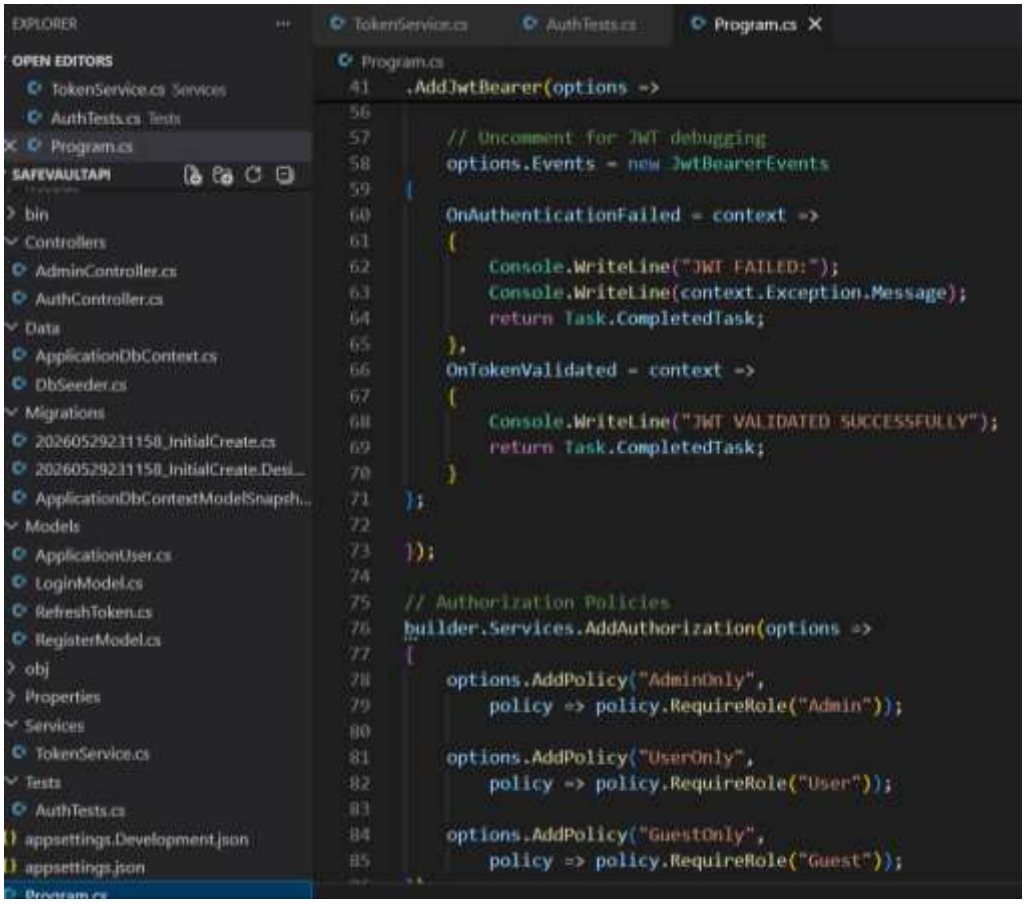


This is where the identity is configured to enforce strong password for security.

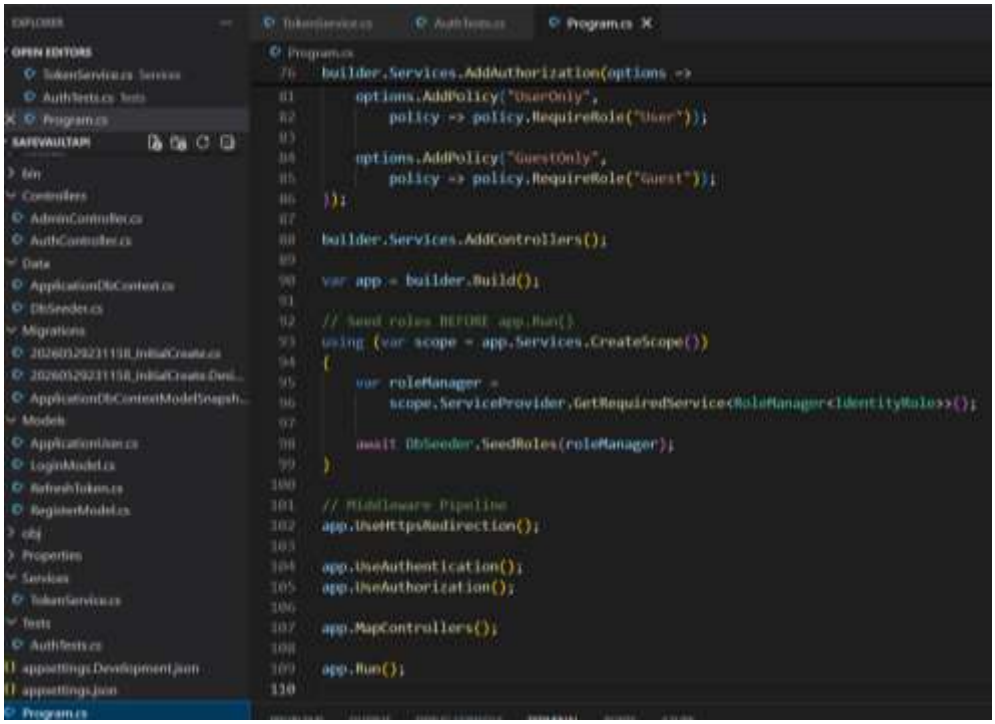




This is where the authorisation policy policies are created.

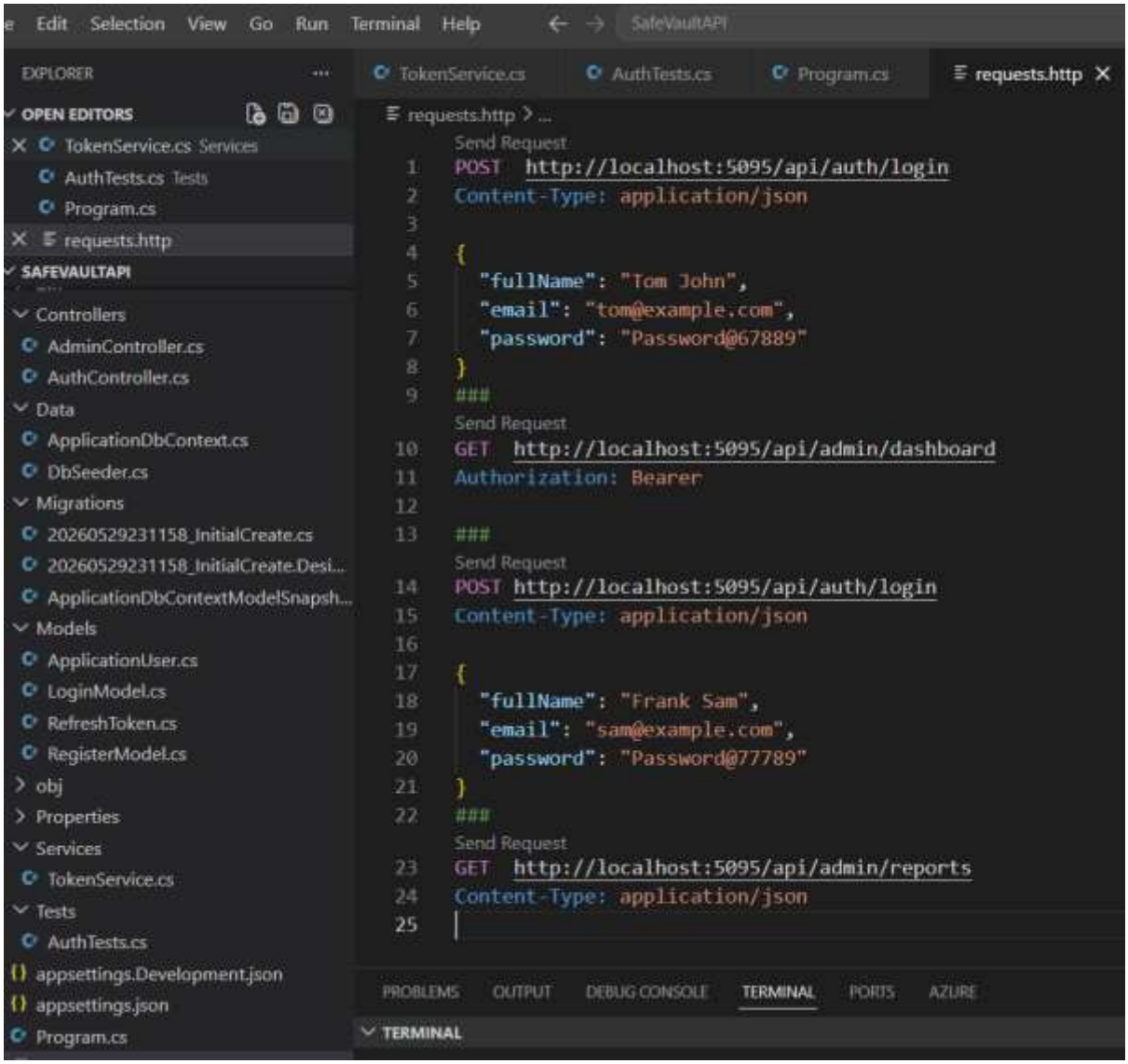


This is the program where roles are seeded and the pipeline for authentication and authorisation.



TESTING

This is where I tested the application my making API calls .


Edit Selection View Go Run Terminal Help
SafeVaultAPI
EXPLORER
TokenService.cs
AuthTests.cs
Program.cs
requests.http
OPEN EDITORS
TokenService.cs Services
AuthTests.cs Tests
Program.cs
requests.http
SAFEVAULTAPI
Controllers
AdminController.cs
AuthController.cs
Data
ApplicationDbContext.cs
DbSeeder.cs
Migrations
20260529231158_InitialCreate.cs
20260529231158_InitialCreate.Desi...
ApplicationDbContextModelSnapsh...
Models
ApplicationUser.cs
LoginModel.cs
RefreshToken.cs
RegisterModel.cs
obj
Properties
Services
TokenService.cs
Tests
AuthTests.cs
appsettings.Development.json
appsettings.json
Program.cs
requests.http > ...
Send Request
POST http://localhost:5095/api/auth/login
Content-Type: application/json
{
"fullName": "Tom John",
"email": "tom@example.com",
"password": "Password@67889"
}
###
Send Request
GET http://localhost:5095/api/admin/dashboard
Authorization: Bearer
###
Send Request
POST http://localhost:5095/api/auth/login
Content-Type: application/json
{
"fullName": "Frank Sam",
"email": "sam@example.com",
"password": "Password@77789"
}
###
Send Request
GET http://localhost:5095/api/admin/reports
Content-Type: application/json
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE
TERMINAL